

课程笔记

Agentic 工作流：从 Workflow 到 Agentic System 的有机融合

视频笔记整理

2026 年 4 月 10 日



视频作者/频道：沧海九粟

发布日期：2025-05-09

视频时长：24:57

视频链接：<https://www.bilibili.com/video/BV1nj5MzyE73>

目录

1 前言	2
1.1 本章小结	4
2 Agent 和 Workflow 概念回顾	4
2.1 经典 AI Agent 的三元组件	4
2.2 传统确定性工作流的定义	5
2.3 LLM: 连接 Agent 与 Workflow 的共同基石	6
2.4 Agentic Workflow: 自主性与确定性的有机中间地带	7
2.5 从 Agent 三元组到 Workflow 三要素的映射	8
2.6 本章小结	8
3 Agentic workflow 核心组件	8
3.1 规划: 任务分解作为固定阶段	9
3.2 工具使用: 预设的查询—选择—执行—响应链路	9
3.3 反思: 显式质检循环与记忆的可选性	10
3.4 纯 Agent 自主性与工作流确定性的对比	12
3.5 本章小结	13
4 两个抽象用例	13
4.1 Agentic RAG 工作流	14
4.1.1 传统 RAG: 一条线性的确定性工作流	14
4.1.2 Agentic 增强: 规划、工具与反思	14
4.2 Agentic 编程助手 (TDD 风格)	15
4.2.1 瀑布式的代码生成流程	16
4.2.2 Agentic 增强: 规划读取、MCP 工具与沙盒反射	16
4.3 本章小结	17
5 两个现实案例	17
5.1 Clay: 数据丰富	18
5.2 ServiceNow: IT 运营支持	19
5.3 本章小结	20
6 总结与延伸	22
6.1 Agentic 工作流的优势	22
6.2 何时不该用: 过度工程的警告	22
6.3 LangGraph 的再定位	23
6.4 回归光谱: 从 Workflow 到 Agent	23
6.5 拓展阅读	24

1 前言

近年来，人工智能领域对 Agent 的讨论热度陡升，尤其是 Anthropic 发布《Building Effective Agents》一文后，“Workflow”与“Agent”这两个概念成为技术社区争论的焦点。然而，许多讨论将二者置于对立面，仿佛必须要在“确定性流程”与“完全自主”之间做出非此即彼的选择。笔者认为，这种二元对立本身就是一种误解。本章将借助 Weaviate 团队对相关主题的延伸阐释，梳理 Agentic 工作流诞生的背景，并提出全书的核心视角：Workflow 与 Agent 并非对立，而是自主性光谱上的不同等级。

背景知识：两篇关键文献

- **Anthropic —《Building Effective Agents》**：首次系统提出 Agentic System 的架构分层，将自动化系统按自主性强度区分为 Workflow 与 Agent 两种形态。
- **Weaviate —“What are Agentic Workflows”**：在 Anthropic 框架基础上进一步扩展，强调如何用大语言模型（LLM）为传统工作流赋能，并总结出 Planning、Tool Use、Reflection 三大增强模式。

在上述文献的语境中，Agentic System（智能体系统）是一个更大的范畴。它既包含高度自由决策的 AI Agent，也包含步骤明确、边界清晰的 Workflow。两者共享同一套底层能力组件——如工具调用（Tool Calling）、检索记忆（Retrieval / Memory）以及推理反思（Reasoning / Reflection）——区别仅在于自主性的强弱。正如视频中所强调的，“它们本质上都带有自动化属性，只是自动化的强弱关系不同”。

核心论点：自主性光谱

Workflow 与 Agent 不是对立阵营，而是**同一家族内部的不同自动化等级**。最恰当的类比是自动驾驶的分级体系：

- **L2（确定性 Workflow）**：人类主导，系统按预设规则逐步执行；
- **L3（Agentic Workflow）**：系统可在固定流程内自主规划、选择工具并反思纠错；
- **L4（Fully Autonomous Agent）**：系统拥有开放目标与持续决策权，人类仅做高层监督。

这种光谱视角是理解后续所有概念与案例的钥匙。

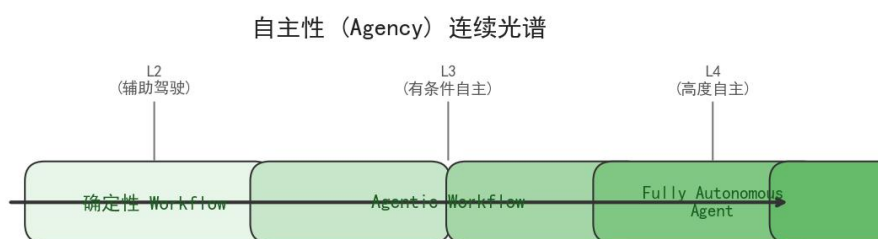


图 1: 自主性光谱：从确定性工作流到完全自主 Agent

不可否认的是，完全自主的 Agent 极具吸引力——它能够根据环境变化自主调整策略、切换路径，甚至重新定义目标。然而，我们必须回到日常业务的现实：大多数企业场景本质上仍是工作流驱动的。无论是客户服务工单处理、数据清洗与归档，还是代码审查发布流程，人类组织的行为模式更倾向于”结构化步骤”而非”开放式探索”。因此，真正具有高商业落地价值的问题，并不是”如何用 Agent 取代 Workflow”，而是”如何让 Workflow 具备 Agent 的能动性”。

常见误区

许多初学者在接触 Agentic 概念后，容易陷入一种技术浪漫主义：认为越自主的系统越先进，从而低估甚至否定结构化流程的价值。实际上，盲目追求”纯 Agent”往往会导致可控性下降、成本不可预测以及合规风险上升。对于生产环境而言，**在确定性框架内注入适度的智能**，通常比放飞完全的自主性更加务实和高效。

基于这一判断，本书后续章节将围绕以下问题展开：大语言模型究竟通过哪几种方式赋能传统工作流？Agentic Workflow 的核心组件是什么？在抽象的 RAG 与编程助手场景，以及 Clay 和 ServiceNow 这样的真实企业案例中，这种赋能如何具体体现？在回答这些问题之前，下一章将先回顾 Agent 与 Workflow 的基础定义，为后续讨论奠定共同的概念地基。

⁰视频画面时间区间：00:00:20–00:01:00。



图 2: Weaviate 原文与白皮书入口

1.1 本章小结

本章作为开宗明义之篇，完成了三项任务：

1. **定位背景**：指出当前关于 Workflow 与 Agent 的讨论热度源于 Anthropic 文章，并由 Weaviate 团队做了进一步延伸；
2. **确立核心视角**：Workflow 与 Agent 不是对立概念，而是 Agentic System 内部因自主性强度不同而形成的光谱，最形象的类比是自动驾驶的 L2/L3/L4 分级（见图 1）；
3. **明确价值主张**：在日常业务中，Workflow 占据主导地位，因此”让工作流 Agentic 化”比”用纯 Agent 取代工作流”更具现实价值。

这一光谱视角将在第二章得到更严谨的概念支撑，并成为全书组织结构的纲领。

2 Agent 和 Workflow 概念回顾

在讨论 Agentic 工作流之前，必须先把两个看似对立的概念——AI Agent 与确定性 Workflow——各自的定义和边界厘清。本节将回顾视频中提出的核心概念：经典 AI Agent 的三大组件、传统确定性工作流的特征，以及为何大语言模型（LLM）的能力是连接二者的共同基石。这些内容为后续引入「Agentic Workflow」打下理论基础。

2.1 经典 AI Agent 的三组件

按照 Weaviate 团队的阐述（并与 Anthropic 在 *Building Effective Agents* 中的提法一脉相承），一个高度自主的 AI Agent 至少具备以下三项核心能力：

- **规划 (Planning)**：将复杂任务分解为子任务，并在执行过程中不断进行推理、反思与调整。

⁰视频画面时间区间：00:01:45–00:01:58。

- **工具使用 (Tool Use / Tool Calling)**: 根据目标自主选择、调用并执行外部工具，将工具的输出纳入后续推理。
- **记忆 (Memory)**: 维护短期上下文（如当前会话历史）与长期知识（如用户偏好、经验积累），以支撑连续的自主决策。

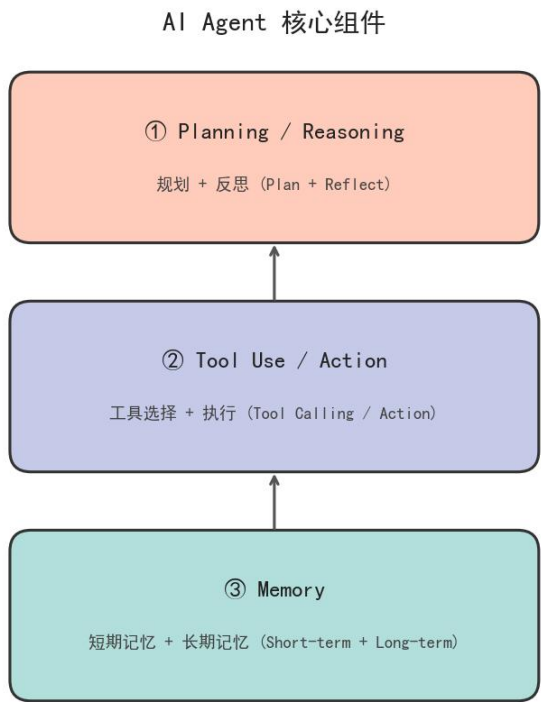


图 3: 经典 AI Agent 的三大核心组件：规划/推理、工具使用、记忆

Agent 的定义要点

一个高自主化 Agent 的运作方式，强调的是通过**自主权**来完成最终目标；Planning、Tool Use 与 Memory 是实现这种自主性的最小必要能力集合。

2.2 传统确定性工作流的定义

与 Agent 相对，传统 Workflow 的核心特征可以用一句话概括：

确定性工作流 (Deterministic Workflow)

为了实现**特定任务目标**而设计的一系列**连续步骤**；最简单的工作流是**确定性的**——流程高度可控、步骤预先设定。

⁰视频画面时间区间：02:32-02:52。

换句话说，传统工作流不关心「自主」，而关心「可控」。它的价值在于把业务过程拆解成固定、可预期的步骤，例如：数据注入 → 召回 → 生成。这种线性或分支结构易于审计、调试和规模化部署。



图 4: 传统确定性工作流示例：步骤 A → 步骤 B → 步骤 C 的线性执行

2.3 LLM：连接 Agent 与 Workflow 的共同基石

视频中反复强调，Agent 与 Workflow 并非对立，而是属于同一个大家庭——Agentic 系统。促成这种统一视角的关键，在于 Anthropic 提出的 **enhanced model / building block** 概念：

- 现代 LLM 不仅仅是文本生成器，它还内建或被增强了一系列能力：**Tool Calling**、**Retrieval**、**Memory**、**ReAct 式推理**。
- 这些能力**绑定在模型上**，无论工程师最终构建的是「工作流」还是「Agent」，都可以复用同一套模型能力。

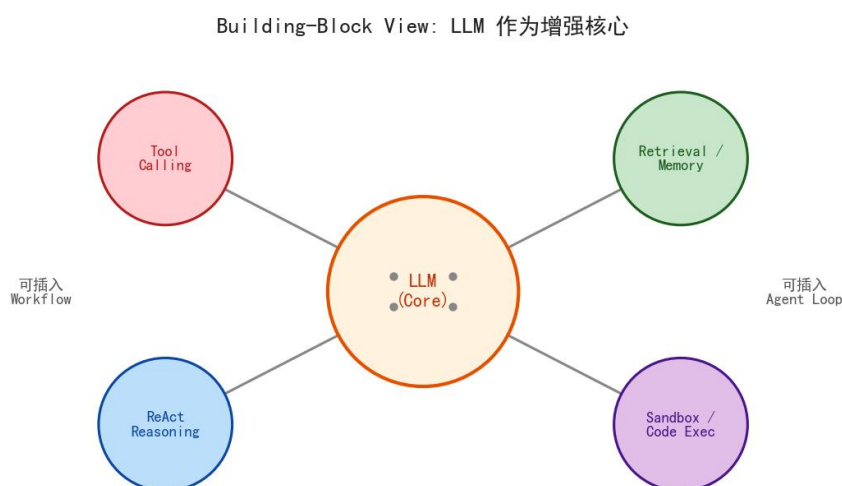


图 5: LLM 增强能力作为公共基础模块：可被 Workflow 线性编排，也可被 Agent 循环调用

⁰ 视频画面时间区间：03:42–03:58。

⁰ 视频画面时间区间：05:08–05:28。

类比：乐高积木

可以把 LLM 的这些增强能力想象成同一套乐高积木。用它们可以搭出一条固定的流水线 (Workflow)，也可以搭出一个能自我进化的机器人 (Agent)。积木本身没有变，变的是组合方式与自由度。

2.4 Agentic Workflow：自主性与确定性的有机中间地带

在上述共识之上，Weaviate 团队给出了 Agentic Workflow 的定义：

Agentic Workflow

由一个或多个 **AI Agent 动态执行**、为实现特定任务目标而构成的一系列连续步骤。Agent 经用户授权，可自主收集数据、执行任务或做出决策，但**整体仍在规划好的结构之中**。

这一定义揭示了三个关键特征：

1. **结构仍在**：和纯 Agent 的「自由发挥」不同，Agentic Workflow 保留了预先设计的步骤框架（先做什么、后做什么）。
2. **节点活化**：工作流中的每个步骤节点，可以由具备 Planning、Tool Use 等能力的 LLM（甚至 Mini-Agent）来执行。
3. **可预期与灵活性并存**：因为流程是预设的，所以仍可追踪、可调试；因为节点是 Agentic 的，所以每个步骤内部能处理更复杂的输入与变化。

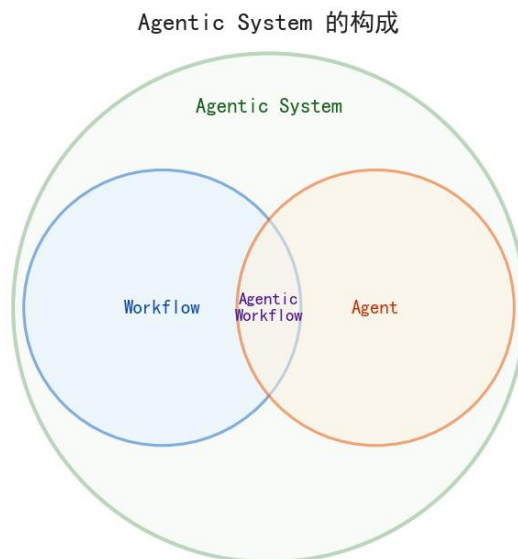


图 6: Agentic 系统：Workflow 与 Agent 不是对立的两个阵营，而是同一光谱上的不同位置；Agentic Workflow 位于二者的重叠区域

⁰ 视频画面时间区间：06:00-06:25。

避免对立思维

讲者指出，当前很多讨论把 Workflow 与 Agent 刻意对立，其实是为了帮助理解概念；但从工程实践看，二者是**你中有我、我中有你的有机整体**。Workflow 的节点可以是 Agent，Agent 的某些执行段也可以用 Workflow 加强其确定性。

2.5 从 Agent 三元组到 Workflow 三要素的映射

视频中进一步将经典 Agent 的三元组件映射到 Agentic Workflow 的三个必备属性上：

- **Planning (制定计划)**：对应 Agent 的任务分解与推理能力。
- **Tool Use (工具执行)**：对应 Agent 的 Tool Calling 能力。
- **Reflection (反思与迭代)**：对应 Agent Reasoning 中的自我纠错；在 Workflow 场景下，它替代了 Memory 成为第三项核心能力。

关于 Reflection 取代 Memory 的原因，speaker 解释得很清楚：Memory 更多服务于 Agent 的开放式自主行动——需要记住上下文才能决定下一步做什么；而 Workflow 的流程是预先确定的，**Memory 并不能改变流程**，因此它是「可选项」而非「必选项」。相比之下，Reflection 能确保固定流程在每一步都产出高质量结果，如果未通过质量检查，则按设计规则重新执行——这对 Agentic Workflow 更为重要。

2.6 本章小结

- 经典 AI Agent 的基石是 Planning、Tool Use 和 Memory；其核心特征是**高自主性**。
- 传统 Workflow 的基石是**确定性步骤**，强调流程可控、目标明确。
- LLM 的增强能力（Tool Calling、Retrieval、ReAct 推理）是**绑定在模型上的公共组件**，构建 Workflow 或 Agent 时均可复用。
- Agentic Workflow 是二者的**有机中间地带**：它保留了工作流的结构，又赋予每个节点以 Agent 的动态执行能力。
- 在 Agentic Workflow 的语境中，**Planning、Tool Use 与 Reflection** 是三项必备属性；Memory 降为可选项。

下一节将围绕这三项属性展开，详细说明它们如何在 Agentic Workflow 中落地为具体的设计模式。

3 Agentic 工作流核心组件

Weaviate 团队在文中提出，使传统工作流具备 Agentic 属性的核心能力有三项：规划(Planning)、工具使用(Tool Use)以及反思与迭代(Reflection / Iteration)。本节将逐一阐释这三项能力在 Agentic 工作流中的具体形态，并重点说明它们与纯自主 Agent 的差异。特别是，我们会解释为何在 workflow 语境下，“反思”取代了传统 AI Agent 三元组中的“记忆”，成为必不可少的第三项能力。

3.1 规划：任务分解作为固定阶段

在纯 AI Agent 的语境中，Planning 指的是 Agent 自主地将复杂任务拆分为更小、更易管理的子任务的能力。这一过程通常与推理和反思交织在一起，Agent 可能会在运行过程中不断地重新调整计划。

而在 Agentic 工作流中，Planning 被固化为一个明确的阶段，即“先规划，后执行”。工作流不会允许 LLM 在执行阶段随意更改整体计划；相反，它会在一个固定的节点上完成全部的任务拆解，然后将产生的子任务序列传递给后续的执行节点。

规划模式的关键特征

- **固定阶段：**规划与执行是两个界限分明的步骤，而不是一个连续的、开放式的思考—行动循环。
- **降低认知负荷：**通过预先分解，LLM 在单个步骤中需要处理的信息量和推理深度都大幅减少，从而提升稳定性、降低幻觉风险。
- **流程可预期：**任务拆解的结果可以作为中间产物被审计、修正或缓存，便于工程化落地。

如图 7 所示，用户输入首先进入一个“Planning”节点，在那里被拆分为子任务 A、B、C；随后这些子任务并行或串行地进入执行阶段。对 Agentic 工作流而言，“规划后执行”是一套固定的两阶段衔接，而非 Agent 那种可能随时根据环境反馈自我修正的开放式行为。

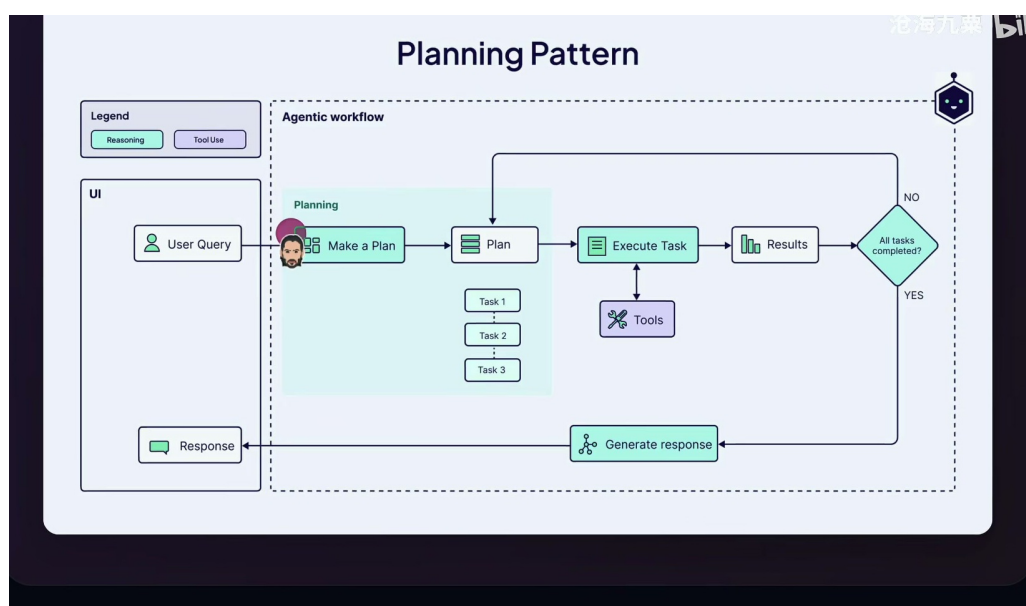


图 7：规划模式：将复杂任务拆分为子任务后，按固定顺序进入执行阶段。

3.2 工具使用：预设的查询—选择—执行—响应链路

Tool Use（或称 Tool Calling）是 AI Agent 与外部世界交互的主要方式。无论是纯 Agent 还是 Agentic 工作流，LLM 都需要具备“理解用户意图 → 选择合适工具 → 调用并获取结果 → 整合结果并生成回复”的能力。然而，这一能力在工作流中的体现形态与在自主 Agent 中有着本质区别。

⁰ 视频画面时间区间：00:09:05–00:09:20。

workflows 中的工具使用 vs. ReAct 循环

- **Agentic 工作流**：工具使用被封装为一条预设的线性链路——用户查询 → 工具选择 → 工具执行 → 生成响应。流程一旦走完即结束，不存在隐式的自动重试或循环机制。
- **纯 ReAct Agent**：决策是动态闭合的。如果工具返回的结果不理想，Agent 会自行决定再次选择其他工具、修改参数，甚至改变策略。选择、执行、观察、推理构成一个自循环。

如图 8 所示，Agentic 工作流中的 Tool Use 模式是一条清晰的单向管道。如果执行结果不满意，工作流不会自动回到”工具选择”节点再次尝试；除非设计者显式地在流程图中添加一条反馈回路（而这正是下一节”反思”要解决的问题）。

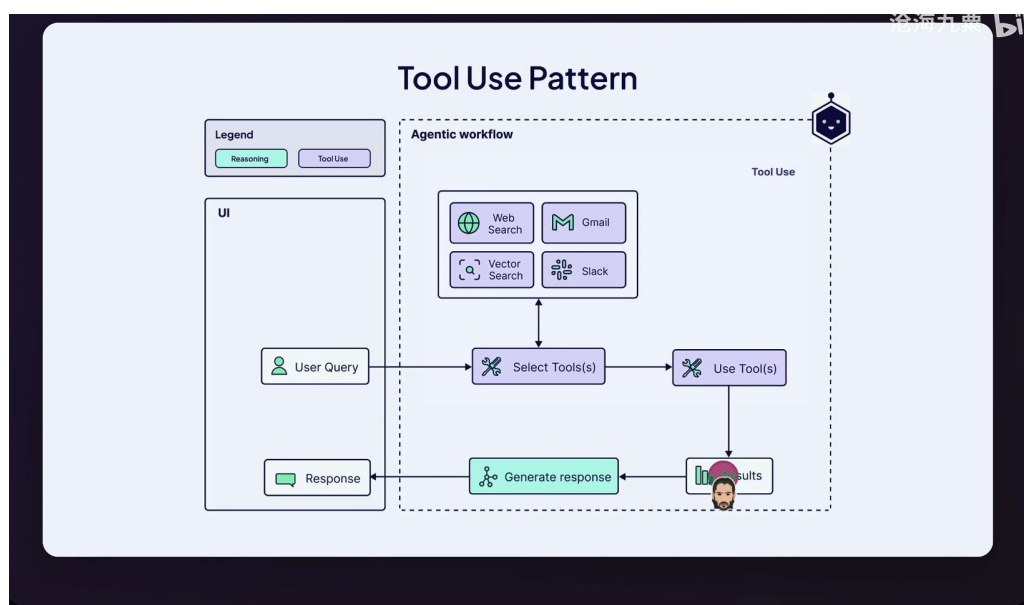


图 8: 工具使用模式：查询、选择、执行、响应构成一条预设链路，无隐式自动重试。

这种”固定性”恰恰是工作流的优势：它保证了流程的可解释性和可预测性。工程师可以明确知道在何种状态下系统会调用何种工具，并且能够对每一步的中间结果进行检查与拦截。

3.3 反思：显式质检循环与记忆的可选性

在传统 AI Agent 的三元组（Planning、Tool Use、Memory）中，Memory 被视为支撑长期自主行为的核心组件。然而，Weaviate 在讨论 Agentic 工作流时，将 Memory 放到了一个相对次要的位置，并用”反思与迭代”取而代之。这一替换绝非随意，而是深刻反映了 workflow 与 agent 在运行范式上的根本差异。

为何用反思替代记忆？

Memory 的主要作用是让 Agent 在开放的环境中保持上下文连续性，从而做出依赖于历史经验的自主决策。但工作流的核心特征是流程的确定性：步骤之间的流转方式是预先设计好的，不会因 LLM 的”记忆”而改变控制流。

⁰ 视频画面时间区间：00:11:00–00:11:15。

因此，对于 Agentic 工作流而言，Memory 是一个有用的**可选项**（例如在多轮对话中保持用户上下文），却不是决定工作流能否高质量完成任务的**必要项**。真正必要的是：在关键节点之后，系统能够**显式地检查自身产出**，并在未通过质检时按照预设规则回溯或重试。这就是 Reflection 的核心价值。

工作流的退出风险

由于 Agentic 工作流的执行节点可能很短（例如只做一次工具调用、生成一段代码或写一行回答），如果中间产物存在问题，流程仍可能径直走到终点并退出。反思步骤的存在，就是为了“不轻易地”放行”这些有瑕疵的结果。

图 9 展示了反思模式在工作流中的典型结构。

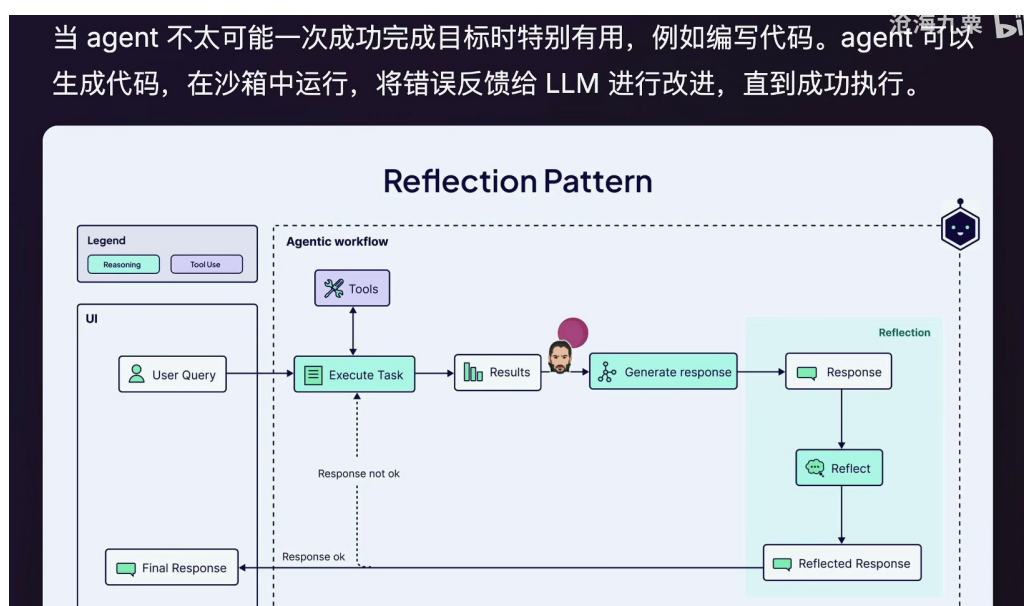


图 9: 反思模式：执行并生成响应后，进入一个显式的 Reflection 节点；未通过则按规则循环回执行阶段。

反思循环的伪代码示例

下面给出一段简明的伪代码，用以展示反思节点如何在预设流程中实现受控的循环：

```
1 planned_steps = planning(user_query)
2 max_retries = 3
3
4 for attempt in range(max_retries):
5     for step in planned_steps:
6         execute(step)
7     result = generate_response()
8
9     if reflect(result) == "pass":
10         return result
11 else:
```

⁰视频画面时间区间：00:12:30-00:12:45。

```

12     log_feedback(reflection_notes)
13     continue # 按预设规则重新执行
14
15 return escalate_to_human()

```

Listing 1: Agentic 工作流中的显式反思循环

这段代码的关键在于：

1. 反思是一个**显式节点** (`reflect(result)`)，而不是 LLM 自由隐式进行的内部思考；
2. 循环的触发条件和最大重试次数由开发者**预先设定**；
3. 若循环耗尽仍未通过，工作流会按规则进入人工兜底或降级处理，而不是让 LLM 自主“想别的办法”。

3.4 纯 Agent 自主性与工作流确定性的对比

通过前三节的讲解，我们已经可以清晰地勾勒出纯 Agent 与 Agentic 工作流在行为范式上的分野。视频作者在此打了一个非常贴切的比方：工作流就是“你告诉 AI 第一步、第二步、第三步怎么做，做完之后再反思，反思不对就回到原点重新做一遍”；而纯 Agent 则是“AI 自己决定怎么做”。

维度	纯 Agent（高自主性）	Agentic 工作流（结构 + 智能）
计划生成	动态生成，执行中可自我修正	固定前置阶段：先规划，后执行
工具调用	ReAct 式自循环，可无限迭代	预设链路，单次通过，不自动重试
错误处理	依赖上下文自主判断新策略	按预设规则回溯或重试（最多 N 次）
结果质检	隐式反思，决策过程不透明	显式 Reflection 节点，可审计
流程可解释性	较低（行为由模型实时决定）	较高（流转路径预先定义）

表 1: 纯 Agent 与 Agentic 工作流的核心对比

图 10 进一步以图文形式展示了同一种“解析网页失败”的场景下，两者的行为差异：

- **纯 Agent**：在两次尝试解析失败后，Agent 可能基于累积的上下文自主决定放弃解析，直接向用户索要其他格式的文件。
- **Agentic 工作流**：流程会严格按照设计执行“解析 → 反思 → 重试”的循环，最多 N 次；若仍失败，则执行预先定义的人工兜底或报错分支，不会自发改变策略。

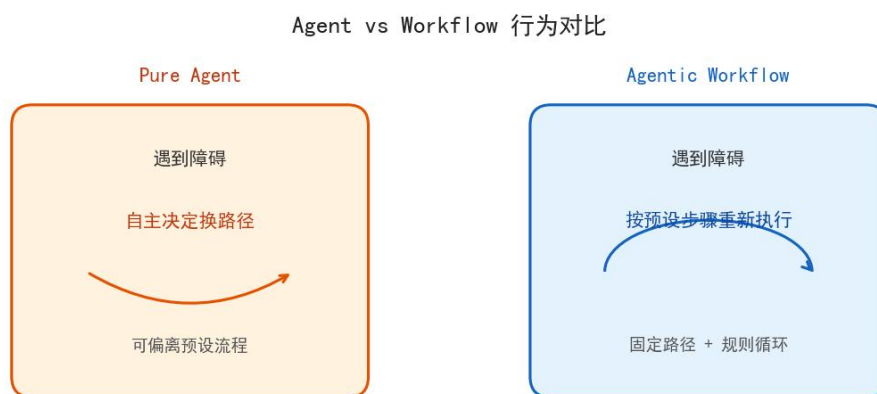


图 10: 纯 Agent（左）与工作流（右）在应对同一失败场景时的行为差异。

正因如此，Agentic 工作流不是对 Agent 的削弱或简化，而是一种在不同场景下更可控、更易落地、更契合现有业务流程的智能化形态。两者并非对立：工作流的节点可以由 Agent 驱动，而 Agent 的某些关键子过程也完全可以被工作流”硬化”为确定性步骤。

3.5 本章小结

Agentic 工作流之所以区别于传统确定性工作流，在于它系统性地嵌入了三种 Agent 能力，同时又将它们约束在预设的结构之中：

1. **规划**：任务分解不再是 Agent 的自由行为，而是被固化为”先规划、后执行”的固定阶段，产出可审计的子任务列表。
2. **工具使用**：查询 → 选择 → 执行 → 响应构成一条预设链路，没有隐式的自动重试循环，确保流程清晰可控。
3. **反思与迭代**：显式质检节点取代了传统 Agent 三元组中的”记忆”，成为工作流语境下保障输出质量的必要机制。失败时按规则循环，而不是让模型自主变更策略。

由此可见，Agentic 工作流与纯 Agent 之间最大的差异在于**自主性与确定性之间的权衡**：纯 Agent 追求最小化人工干预、最大化自适应；而 Agentic 工作流则在保留 LLM 智能的同时，将流转路径外显为可设计、可监控、可维护的结构。这种”你中有我、我中有你”的关系，正是它们同属 Agentic System 这一更大谱系的明证。下一节，我们将看到这三项核心能力如何在两个典型的抽象用例——Agentic RAG 与 Agentic 编程助手中——得到具体应用。

4 两个抽象用例

前面我们讨论了 Agentic 工作流的三项核心属性——规划、工具使用与反思迭代。为了更具体地理解这些属性如何在实际场景中落地，本节选取了两个最具代表性的抽象用例：Agentic RAG 工作流与 Agentic 编程助手（以 TDD 风格为例）。这两个例子分别覆盖了信息检索领域与代码生成领域，能够清晰地展示“固定流程 + 智能化节点”的有机融合方式。

4.1 Agentic RAG workflow

检索增强生成 (RAG) 本身天然符合工作流的特征：它有明确的输入、召回与生成三个阶段，数据流向清晰，步骤之间因果关系稳定。因此，当我们讨论如何为 RAG 注入 Agentic 能力时，最典型的做法并不是把它拆成一个完全自由的 Agent，而是在这条预设流程的特定节点上加入规划、工具使用与反思。

4.1.1 传统 RAG：一条线性的确定性 workflow

在没有引入 Agentic 增强之前，传统 RAG 的流程通常被描述为以下三个步骤：

1. **数据注入**——将原始文档切分、嵌入并写入向量数据库；
2. **检索召回**——根据用户提问从向量库中召回 top- k 个最相关的片段；
3. **上下文增强生成**——将召回的片段与用户问题一同送入大语言模型，生成最终答案。

这是一个典型的线性 workflow：信息从左到右单向流动，没有循环，也没有额外的决策节点。对于结构化的内部文档、问答对明确的场景，传统 RAG 已经能够取得不错的效果。

然而，当用户的问题变得复杂、跨文档、或需要结合实时外部信息时，传统 RAG 的局限性便会显现：

- **检索质量不佳**——单一向量召回容易漏掉关键信息，或者召回大量无关内容；
- **工具链路单一**——只依赖向量存储，缺少对网页搜索、结构化数据库、API 等其他信息源的调用；
- **缺乏验证环节**——生成结果直接返回给用户，没有针对答案准确性与召回充分性的自我检查。

4.1.2 Agentic 增强：规划、工具与反思

Agentic RAG 的核心理念，就是在保留“注入 → 检索 → 生成”这条主流程的前提下，为每个阶段注入智能化的判断与动作。图 11 展示了这一架构的示意。

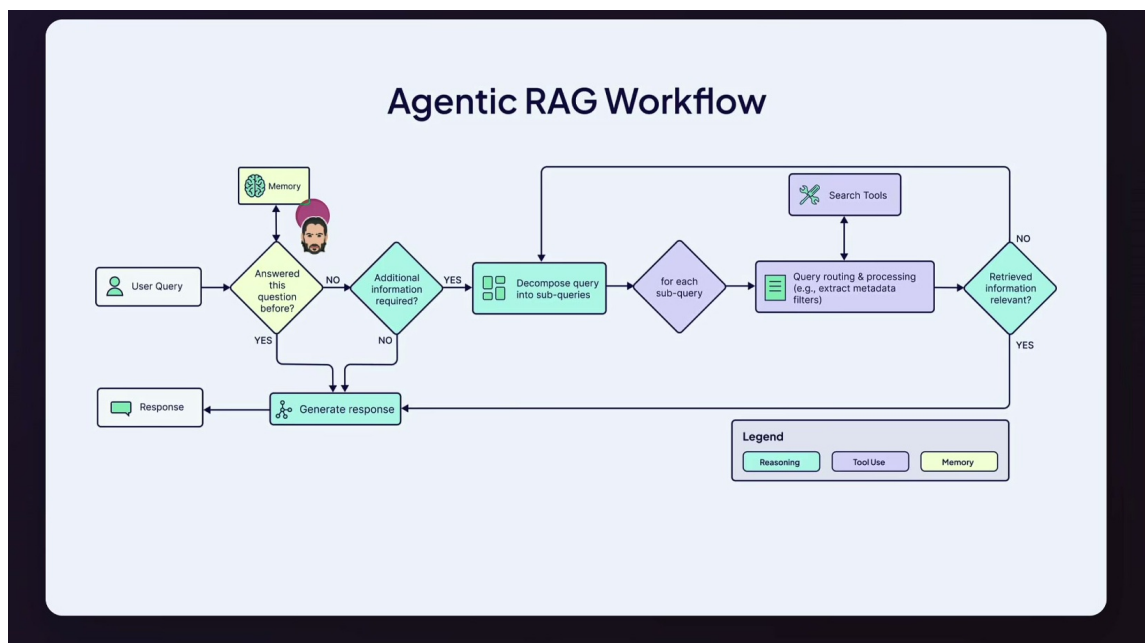


图 11: Agentic RAG workflows 架构：在传统 RAG 的三段式流程上叠加规划（子查询拆解）、工具使用（向量 + 网络搜索）与反思（质量门禁）三层 Agentic 增强。

Agentic RAG 的三层增强

1. **规划 (Planning)**：在检索之前，先将用户问题拆分为若干子查询。每个子查询可以对应不同的检索策略或数据源，从而降低单次检索的认知负荷，提高召回的全面性。
2. **工具使用 (Tool Use)**：不仅调用向量数据库，还可以根据规划结果触发网络搜索 (Web Search)、结构化查询或其他外部 API。当向量存储不足以回答问题时，工具链路的扩展能够显著增强系统的信息获取能力。
3. **反思 (Reflection)**：在生成答案之后设置一个显式的质量门禁。系统会自我评估检索结果是否充分、答案是否准确；若未通过，则按照预设规则回退到检索或生成阶段重新执行。

从 workflow 的角度看，上述增强并没有破坏 RAG 的线性骨架，而是在特定节点插入了“智能化子程序”。例如，反思失败后的回退路径是预先设计好的：如果答案质量不达标，可以重新执行检索（更换关键词或扩展数据源），或者重新生成（使用更严格的提示词）。这与完全自主的 Agent 有本质区别——后者可能会自行决定放弃原有策略、转向全新的解决路径，而 Agentic workflow 中的回退路径是可控且可预期的。

4.2 Agentic 编程助手 (TDD 风格)

软件工程天然具有流程化的特征。以测试驱动开发 (TDD) 为例，开发过程可以被清晰地划分为：阅读现有代码 → 编写测试 → 编写实现 → 运行验证 → 反思修正。这一瀑布式的步骤结构，使其成为展示 Agentic workflow 优势的绝佳案例。

⁰ 视频画面时间区间：00:16:00–00:16:15。

4.2.1 瀑布式的代码生成流程

如果我们把编程助手看作一个工作流，它的传统版本可能是这样的：

- 1. **Context Read**：读取代码库中相关的文件与上下文；
- 2. **Test Write**：根据需求编写测试用例；
- 3. **Code Write**：编写通过测试的实现代码；
- 4. **Execution**：在沙盒或解释器中运行代码与测试；
- 5. **Reflection**：检查运行结果，若存在错误则回到前面的步骤进行修改。

这个流程的每一步都是明确的，最终目标是产出“可运行、可验证”的代码。然而，传统编程助手往往只做到第 3 步——即代码生成——然后就直接将结果交给用户。用户需要手动复制代码、运行测试、定位错误，再回头要求模型修改。Agentic 编程助手的目标，就是把第 4、5 步也纳入自动化工作流，并赋予模型在每一步中自主决策的能力。

4.2.2 Agentic 增强：规划读取、MCP 工具与沙盒反射

图 12 展示了 Agentic 编程助手的水瀑式工作流。与 Agentic RAG 类似，它的 Agentic 增强同样体现在三个维度：

沧海九粟

bilibili

</> Agentic 编程助手

能够以最少的人工干预生成、重构、改进和调试代码。非 agentic 助手（如旧版 GitHub Copilot）仅限于生成代码。

Agentic 助手的特点是能够通过执行生成的代码与环境交互，并根据执行结果、错误或反馈迭代改进。它们可以被授权通过创建 commit 和 PR 来更改现有代码库。

功能	Agentic 编程助手	传统编程助手
代码生成	✓	✓
代码执行	✓	✗

图 12: Agentic 编程助手工作流：Context Read → Test Write → Code Write → Sandbox Execution → Reflection，失败时按预设规则循环回退。

规划（Planning）——智能文件读取 并非一次性把整个代码库塞进上下文，而是让模型先判断“哪些文件与当前任务最相关”，并按优先级逐个读取。这种规划能力能够显著降低上下文窗口的压力，同时避免无关信息干扰生成质量。

⁰ 视频画面时间区间：00:17:30–00:17:45。

工具使用 (Tool Use) ——mcp 与沙盒 现代编程助手可以通过 MCP (Model Context Protocol) 接入丰富的外部工具：代码搜索、文档查询、依赖分析、LSP 补全等。在 Execution 阶段，模型可以把生成的代码送入 Code Interpreter 或隔离沙盒中运行，获取真实的执行反馈（包括报错堆栈、测试覆盖率、性能指标）。

反思 (Reflection) ——基于执行错误的自我纠正 当沙盒返回编译错误或测试失败时，Reflection 节点会分析错误原因，决定是回退到 Code Write（修改实现）、Test Write（调整测试），还是 Context Read（补充遗漏的上下文）。这个循环是设计好的：例如，“最多允许 N 次重试，超过则终止并提示用户”。

从“代码生成”到“可验证交付”

传统编程助手的输出是一段静态文本；Agentic 编程助手的输出则是一段经过规划、执行与反思验证的、可运行的代码。核心差异在于：

- 它不仅“写”代码，还“读”上下文、决定读什么；
- 它不仅“给”代码，还“跑”代码、观察真实执行结果；
- 它不仅“一次生成”，还在失败时按预设路径自动迭代。

需要再次强调的是，这个编程助手仍然是一个工作流，而不是一个完全自由的 Agent。它的整体结构（读 → 测 → 写 → 跑 → 反思）是预先确定的；模型在每个节点内部拥有自主权，但节点之间的流转路径是人类设计好的。这种“固定骨架 + 智能血肉”的组合，正是 Agentic 工作流的精髓所在。

4.3 本章小结

本节通过两个抽象用例——Agentic RAG 与 Agentic 编程助手——具体展示了如何让传统的确定性工作流具备 Agentic 能力。无论是检索领域还是代码生成领域，核心思路都是一致的：

- 保留清晰的主流程与节点顺序；
- 在关键节点注入规划（任务分解、路径选择）、工具使用（多源信息、外部执行环境）与反思（质量检查、失败回退）三项能力；
- 确保循环与回退路径是可控、可解释、可预期的，而非完全交由模型自由发挥。

下一节，我们将把视线从抽象用例转向真实的企业产品，看看 Clay 的数据丰富流程与 ServiceNow 的 IT 运营支持系统，是如何在生产环境中落地 Agentic 工作流理念的。

5 两个现实案例

前面两节讨论了 Agentic 工作流的核心组件与抽象用例。接下来，我们把视角从架构图拉回真实的商业产品，看看两套已经被验证的落地流程：Clay 的数据丰富 (Data Enrichment) 与 ServiceNow 的 IT 运营支持。它们共同印证了一个设计原则——**固定的阶段带来可控性，Agentic 的节点带来智能增量。**

5.1 Clay: 数据丰富

Clay 是一家专注于销售情报与数据丰富的 SaaS 公司。在日常销售工作中，团队往往需要面对海量、分散的潜在客户信息：一份简历、一个 GitHub 主页、一篇 arXiv 论文、一条 LinkedIn 动态……如果全靠人工整理，既耗时又难以保证一致性。

Clay 的解决方案不是做一个完全自由的”通用 Agent”，而是把整条链路封装成一条预设的 Agentic 工作流。如 图 13 所示，这条流水线可以概括为四个核心阶段：

1. **Extract (抽取)**：从用户提供的简历或链接中抽取核心信息（姓名、职位、技能标签、项目经历等）。
2. **Crawl (爬取)**：基于抽取结果中的外链，自动到 GitHub、arXiv、个人博客等来源抓取补充内容。
3. **Analyze (分析)**：把这些原始素材交给大语言模型进行语义分析，生成结构化的洞察（例如技术栈匹配度、研究兴趣摘要）。
4. **Refine (精炼)**：对模型输出做格式校正、去重、质量过滤，最终生成一份干净、可直接导入 CRM 的 enriched profile。

Agentic 的节点在哪里？

在这四个阶段里，

- **规划**体现在 Crawl 阶段：Agent 需要**自主决定**访问哪些外部来源、按什么优先级抓取；
- **工具使用**体现在对 GitHub API、arXiv 接口、网页解析器的动态调用；
- **反思**体现在 Refine 阶段：如果抽取结果格式错乱或置信度过低，系统会回退到前面的步骤重新处理，而不是直接把”半成品”交给用户。

Claygent (来自 Clay)

沧海九粟 

潜在客户研究和数据丰富对增长和销售团队来说可能很繁琐。Clay 公司通过 Claygent 简化了这一过程——这是一个 AI 驱动的研究 agent，可持续扫描网络和内部数据库以提供实时、可操作的见解。

例如，使用 Claygent 基于姓名和电子邮件列表丰富 LinkedIn 个人资料，然后发送个性化介绍信息。流程：

1. 指定所需数据字段（工作经验、教育、技能等），注入预配置的提示模板。
2. Agent 的 LLM 处理查询，使用网页抓取工具搜索 LinkedIn URL。
3. 从 LinkedIn 个人资料中提取所需数据。
4. 数据发送给另一个 LLM 进行总结/分析。
5. 同一个（或另一个）LLM 为每个个人资料创建个性化外联消息。

图 13: Clay 数据丰富工作流：Extract → Crawl → Analyze → Refine。Crawl 与 Refine 步骤体现了 Agentic 增强（规划、工具调用与反思）。

整条流程的方向是固定的（extract → crawl → analyze → refine），但每个节点内部都被 LLM 赋予了有限的自主权。这正是 Agentic 工作流在 B2B SaaS 中的典型落地形态：用确定性流程保证交付边界，用 Agentic 节点提升数据质量与覆盖度。

5.2 ServiceNow：IT 运营支持

如果说 Clay 的案例展示了前台业务如何被 Agentic 流程增强，那么 ServiceNow 的案例则展示了后台运营中的人机协同（Human-in-the-Loop）设计。

ServiceNow 的 IT 运营支持流程如图 14 所示：

1. **Ticket 触发**：客户提交故障工单或 Support Request。
2. **KB 检索**：一个或多个 Agent 被触发，在企业内部知识库（Knowledge Base）中检索相关文档与历史案例。
3. **摘要与案例分析**：Agent 对召回内容进行摘要，并分析历史上相似的故障根因与处理方案。
4. **生成建议**：基于上述信息，为 IT 支持专家（Support Engineer / Expert）生成一份包含摘要、可能根因、推荐操作的处理意见。
5. **人工审批门**：专家可以**批准、修改或拒绝**该建议，再由系统执行最终动作（如自动回复客户、创建变更单、升级二线）。

⁰ 视频画面时间区间：00:19:15–00:19:30。

ServiceNow 是一个简化和自动化 IT、运营、HR 和客户服务领域工作流的云平台。其平台现在包含 AI agent，旨在自动化重复性任务和现有工作流，同时让人类完全掌控决策权。

解决技术支持案例的 agentic 工作流示例：

1. 客户提交技术支持工单，触发工作流。
2. 工单信息传递给一个或多个 agent，对内部 IT 支持知识库执行 RAG。
3. Agent 总结发现，分析类似案例，并为 IT 支持专家生成摘要。
4. 生成处理建议，专家可以批准或拒绝。

ServiceNow AI Agents 代表了一种创新但更谨慎的在生产环境中部署 agent 的方法，赋予它们严格的角色和任务，决策自主权有限或没有。

图 14: ServiceNow IT 运营支持流程：Ticket 触发 → KB 检索 → 摘要与案例分析 → 生成建议 → 人工审批门。

为什么必须保留 Human-in-the-Loop ?

在 IT 运营这种容错成本较高的场景中，完全自主的 Agent 并不现实。ServiceNow 的设计智慧在于：把检索、摘要、归纳这些信息密集型工作交给 Agent，把最终决策权牢牢留在人类专家手中。这种“后台 Agent + 前台人工门”的结构，也被称为 **Background Agent** 或 **Ambient Agent** 模式——Agent 在流程中默默运转，人类只在关键节点介入。

在这套流程中，Agentic 的环节体现在：

- **检索策略**：Agent 可以决定使用哪些知识库索引、是否拆分子查询、是否结合向量检索与关键词检索；
- **摘要生成**：对多源召回内容进行整合、去噪、结构化；
- **推荐生成**：将历史案例映射到当前 ticket，输出可执行的建议。

与传统工单系统相比，这一方案显著缩短了“从问题到建议”的时间，同时通过人工门控制了决策风险。

5.3 本章小结

Clay 与 ServiceNow 两个案例虽然分属不同业务域，但遵循了同一套设计范式：

- **流程骨架是预设的**：数据丰富有第四步，IT 支持有五步，方向不会因为 LLM 的“突发奇想”而改变；

⁰视频画面时间区间：00:21:00-00:21:15。

- **智能增强是局部的：**规划、工具调用、反思只发生在特定节点内，而不是贯穿全程的开放循环；
- **业务约束决定自主性边界：**销售场景允许更高的端到端自动化，IT 运营场景则必须在最后一步引入人类审批。

这套”固定框架 + Agentic 节点”的混合模式，也正是大多数企业从”概念验证”走向”生产落地”时最稳健的选择。

6 总结与延伸

6.1 Agentic 工作流的优势

将 Agent 的能力注入传统工作流，带来的不是单点性能提升，而是整条链路性质的增强。结合视频中的梳理，这些优势可以归纳为六个方面：

1. **灵活性 (Flexibility)**: LLM 可以在节点内部动态调整策略，例如根据输入难度选择不同的工具组合；
2. **可定制性 (Customizability)**: 每个阶段的提示词、retry 规则、fallback 路径都可以按业务需求精细配置；
3. **处理复杂任务的能力**: 通过规划 (Planning) 将大问题拆解为可管理的子任务，降低单次推理的认知负荷；
4. **自我纠正 (Self-Correction)**: 显式的 Reflection 节点让系统在输出前就能发现错误并回退重试；
5. **持续学习能力**: 失败的反射回路可以被记录并用于迭代优化提示词或工具定义；
6. **效率与可扩展性**: 自动化处理大量原本需要人工介入的信息整理、检索、摘要工作。

需要再次强调的是，以上优势是**相对于传统工作流**而言的，而不是相对于”纯 Agent”。Agentic 工作流的价值不在于它比谁都更自主，而在于它比传统工作流更聪明，又比纯 Agent 更可控。

6.2 何时不该用：过度工程的警告

视频中，讲者反复提醒观众不要”为了 Agentic 而 Agentic”。如果以下任一条件成立，强行引入 LLM 反而可能造成过度工程：

- 传统确定性工作流已经能稳定解决问题；
- 传统 RAG 加上简单的 guardrails（如阈值过滤、关键词黑名单）即可满足质量要求；
- 任务对延迟极度敏感，而 LLM 的推理与工具调用会显著拖慢响应；
- 错误成本极高，且当前 LLM 的幻觉率无法接受。

正如图 15 所示，选择是否引入 Agentic 工作流，本质上是一道关于**任务复杂度、纠错需求、错误成本、基线性能**的综合决策题。

挑战与局限

- **对简单任务过度复杂：** 对于简单的、基于规则的自动化就足够的情况，引入 agent 可能导致效率低下、额外成本和性能下降。
- **自主性增加导致可靠性降低：** agent 的概率性可能引入不可预测性，使输出可靠性降低且难以控制。实施和维护护栏 (guardrails) 及持续审查权限至关重要。
- **伦理和实践考量：** 并非所有决策都应委托给 AI。在高风险或敏感领域使用 agent 需要仔细监督，以确保负责任部署并防止意外后果。

反思是否真的需要 agent：任务是否足够复杂？简单的 AI 辅助工具（如无 agent 的 RAG）是否能达到相同效果？工作流是否涉及不确定性、变化条件或多步推理？

图 15: 何时该用 Agentic Workflow? 任务复杂度、纠错需求、错误成本与基线性能是四项关键考量。

6.3 LangGraph 的再定位

在工程实现层面，很多开发者会把 **LangGraph** 直接等同于“Agent 框架”。但视频中的解读更为准确：

LangGraph 本质上是一个**状态图编排框架** (state-graph orchestrator)。你完全可以用它搭建一个不带任何 LLM 的纯确定性工作流：Step A → Step B → Step C，有分支、有循环，但没有 Agentic 元素。只有在需要时，你才把 LLM 节点嵌入其中。

这种定位上的澄清很重要。它提醒我们：工具本身是中性的，关键在于设计者是否清楚自己的流程需要多少自主性。

6.4 回归光谱：从 Workflow 到 Agent

回顾 图 1 中介绍的自主性光谱——

- 左侧是**确定性工作流**（每一步都被写死）；
- 中间是 **Agentic 工作流**（流程固定，节点智能）；
- 右侧是**完全自主的 Agent**（目标给定，路径自决）。

Clay 与 ServiceNow 都稳稳地落在中间地带。它们没有追求自动驾驶 L4 级别的完全无人化，而是选择了 L2/L3 级别的“智能辅助 + 人类兜底”。对于大多数企业而言，这并不是退而求其次，而是在可控性与智能化之间找到的最优点。

6.5 拓展阅读

视频最后强烈推荐了 Weaviate 团队的免费白皮书 *Agentic Architecture for Retrieval-Intensive Applications*。该书系统梳理了 Agentic System 的整体架构，包括 Workflow、Agent、工具层、记忆层与检索层的协作方式，是对本节内容的极佳补充。

延伸阅读：

Weaviate — *What are Agentic Workflows?*

<https://weaviate.io/blog/what-are-agentic-workflows>